

Aussie EcoLens: A Multi-Cloud Serverless Wildlife Observation Platform

FIT5225 2026 S1 Assignment 2 (Weight: 40%)

Due date: Monday of Week 14 (23:55, 8 June)

1 Background

Australia boasts some of the most unique and diverse wildlife on the planet. A machine learning-based, vision-driven platform plays a critical role in citizen science, allowing researchers, students, and nature enthusiasts to capture, log, and identify fauna across the continent. Think about all the fantastic stuff captured: stunning shots of wildlife in action, unique recordings of calls, and insightful video clips of behaviour. This is a gold mine for environmental research, helping to identify species, track individuals, study behaviour, monitor populations, and ultimately protect our ecosystems.

However, imagine trying to find a specific photo of a rare species from last year if it's buried on someone's random hard drive, or losing a whole batch of important video recordings because a memory card got corrupted. Data scattered everywhere is data that's hard to use, easy to lose, and impossible to share properly. A good storage system means everything is safe, backed up, and organised in one central location, ensuring that any member who needs data can find it efficiently. Collaborating on projects becomes a breeze because everyone looks at the same organised data, ensuring valuable information isn't lost.

This assignment aims to build a **multi-cloud** online storage system that allows users to store, organise, and retrieve media files based on **auto-generated** and manual tags. The focus of this project is to design a **serverless** application across **at least two major cloud providers** (e.g., AWS, GCP/Azure/Oracle) that enables clients to upload their media files (**images and videos**) to public cloud storage. Upon upload, the application automatically tags the file with the species detected in it using pre-trained ML models, automatically sorts/moves the file to the corresponding directory, and adds a record in a database for searching.

2 Group Assignment

Most software developers will work in a team at some point in their career. Therefore, this assignment is designed as a group project to prepare students to experience software development as a team. Students will team up in groups of up to 4 to implement the system described above. In this assignment, students need to organise their teams and collective involvement. There is no designated team leader, but teams may establish processes to agree

on the distribution of work.

Understanding the dependencies between individual efforts and their successful integration is crucial to the success of the work and for software engineering projects in general. If teams encounter any "issues", they should try to resolve the issues first. In case of an unresolvable issue, please inform the teaching team as soon as possible.

The due date for this assignment is firm, and there is no 'special consideration' for granting an extension (i.e., individual special consideration or extension doesn't apply to a group). **If a team member fails to complete their work for any reason, other team members should take over their tasks.** We will evaluate individual involvement in the project. If a team member fails to satisfy their tasks for non-compelling reasons, they may be penalised heavily, regardless of the project's success.

2.1 Assignment Description

Teams should develop a multi-cloud solution that leverages serverless compute, RESTful API, storage, and database services across multiple providers (e.g., AWS, GCP, Oracle or Azure) to build a system for automated wildlife tagging and query processing. The teams should produce a solution that enables end-users to upload their files to a cloud storage bucket. Upon uploading a file to the designated bucket, a serverless function is automatically triggered, using the provided model to identify the species in the file and store the list of detected tags, along with the file's storage URL, in a highly available database. Furthermore, the end user should be able to submit queries to a RESTful API endpoint to search for tagged files.

3 Authentication and Authorisation (10%)

Security is one of the most crucial aspects of developing cloud applications. When your application is publicly exposed, you must ensure that your endpoints and resources are safeguarded against unauthorised access and malicious requests. AWS, through its Cognito service, provides a straightforward, secure, and centralised approach to protect your web application.

Note on Multi-Cloud: For user authentication and authorisation, you must use AWS Cognito. Other services can be hosted in other cloud services. **You may need to consider how to securely pass AWS Cognito-generated authentication tokens to authorise requests to your secondary cloud provider's API endpoints or serverless functions.**

Your application should support the following workflow and features:

- Detect whether a user is authenticated. If the user has not signed in, access to all pages and endpoints except the sign-up service should be blocked, and the user should be redirected to the sign-up page to register a new account.
- For each new account, you need to record the user's email address, first name, last name,

and password. Cognito will send an email to new users, asking them to verify their email address and change their temporary password.

- Your application should include a login page that allows users to sign in. After successful authentication, users should be able to upload files, submit queries, view query results, and sign out of the application. All of these services must be protected against unauthorised access.
- You can implement login and sign-up web pages using either the Hosted UI feature in Cognito or your own custom implementation that calls Cognito APIs.
- Uploading files, invoking serverless functions, and accessing the database require fine-grained access control permissions that you need to configure via IAM roles (or equivalent policies in your secondary cloud). It is important to note that IAM roles in AWS Academy have several limitations. Therefore, you should carefully consider how to perform authentication and authorisation in your system while considering these limitations.

4 Core Functionalities (50%)

4.1 Model Handling

Machine learning and deep learning models are often updated to improve their quality. If a new version of the pretrained model becomes available, your solution design should be flexible enough to use it without changing your source code/lambda function.

4.2 File Handling

Your solution should provide a mechanism to upload a file to cloud storage (e.g., an S3 bucket). Uploading a file can be done either through a RESTful API endpoint or directly using cloud SDKs. Whenever a file is uploaded, your system must trigger an event and invoke a serverless function.

- **Deduplication:** Users may upload the same file repeatedly, wasting storage space. Your application should have a mechanism to prevent duplicate uploads using a file checksum.
- **Building Image Thumbnails:** Upon uploading an image, a serverless function should generate a thumbnail for it. Creating a thumbnail involves resizing the original image to a smaller dimension while maintaining its aspect ratio, then compressing it to reduce file size. You can use the OpenCV library for this purpose. You are expected to configure the triggers and grant the required resource permissions (execution roles) for the function.
- **Tagging Files:** Another serverless function should be triggered to read the uploaded file, detect species using the ML model, and save the file type and a list of detected tags, along with the storage URLs for the file and its thumbnail (for images), in a database for future queries.
- **Video files** can be handled by extracting frames as images, then using the image detection model. For consistency and reduction of data processing, please extract 1

image per second (do not try to extract all frames).

4.3 Queries

Your application should provide APIs that allow the following queries:

- **Find images and videos based on the tags:** The user can send a JSON-based query to request URLs of files that contain specific tags with minimum counts (e.g., {"koala": 3}, {"wombat": 2, "magpie": 1}, which shows all files with ≥ 3 koalas in the first case and ≥ 2 wombats AND ≥ 1 magpie in the second example). You are expected to create a RESTful API allowing users to submit requests. A response should include the list of URLs to all thumbnails of images and full URLs for videos that contain all the requested tags. Your query may require triggering a function that receives a list of tags from the database, i.e., logical AND operation, not OR operation between tags. In your UI, you can preview the thumbnails found as the results (to save bandwidth), and users can request the full-size image by clicking on them.
- **Find files based on species:** User can send a request similar to the previous query without count, e.g. {"dingo"}. Your application will return all images and videos containing at least one dingo.
- **Find files based on the thumbnail's URL:** Users can input the URL of a thumbnail into the system and receive the corresponding full-size image URL.
- **Find files based on the tags of a file:** The user can send a file as part of an API call. The list of all species (tags) in the sent file is discovered, and then all the files in the database containing that set of tags are found. As a response, the list of URLs of the matching media files is returned to the user. Ensure that the file uploaded for query purposes is not added to the database or permanently stored.
- **Manual addition or removal of tags with bulk tagging:** Your solution should provide a POST API that enables end-users to manually add or remove tags from files. The JSON message sent must include a list of URLs, the tags to modify, and an "operation" key set to 1 (for adding a tag) or 0 (for removing a tag). If a tag is not included in the list of tags requested for deletion, simply ignore it.
- **Delete files:** The user can send a list of URLs to an API, and the system should remove the files and their thumbnails from storage and all relevant entries from the database.

4.4 Tag-based Notifications

Users can receive email notifications or updates related to files based on specific tags. For example, users can receive notifications when new images with specific species are added or updated to the system. You can use AWS SNS or a similar serverless messaging equivalent on

your secondary cloud for this purpose.

5 User Interface (20%)

A user interface (UI) makes your system easier and more enjoyable to use. You can design a simple web-based UI. The UI must support all core functionalities, including uploading files, submitting queries, viewing query results, adding/removing tags, etc.

You have the option to either not create a UI for the application or to implement a partially implemented UI for some parts. However, if you opt for this, you need to consider how to demonstrate the core functionalities during the demo. Also, without a UI, the maximum mark you'll receive is capped at D (70% or 14 marks in this section).

6 Demo and Final Reports (20%)

You should write an individual and team report on the developed application. Use a highly legible font such as Arial, Helvetica, or Times New Roman with a font size of 12 points.

6.1 Demo Performance (15%)

6.2 Team Report (Maximum 1000 words)

Please prepare a team report based on the following guidelines:

- Include a multi-cloud architecture diagram in the team report. For the architecture diagram, you must use official architecture icons for all involved cloud providers.
- Include a table to describe the role of each team member in your team report. You shall provide a three-column table that shows student's name and ID, the percentage of contribution, and the elements of the project the member contributed (a maximum of 100 words per member).
- The report includes a simple user guide for testing your application.
- Your report should include a link to the source code (GitHub, Gitlab, or Bitbucket). All students must commit their code to the code repository rather than delegate this to a single team member. This can provide an evidence base if teams run into "issues" and show how you contributed. You should share your repository with the whole teaching team. Please do not use a public repository to avoid any plagiarism.
- Note that the tables, architecture diagram, screenshots of UI, and references are excluded from the word count limit.

6.3 Individual Report - 500 words

Please prepare an individual report based on the following guidelines:

- In your individual report, you should describe your role in the project and how you contributed to the delivery of the multi-cloud system (150 words).

- Include a personal reflection on the teamwork experience of working with other individual team members in your group. Comment on your teammates' work, challenges the entire team faced, and how well the team worked (150 words).
- Note that the individual report should be done with little or no consultation with other team members. Please be careful about plagiarism and collusion.

7 Submission

You need to submit the following via Moodle:

- Your **team report** in PDF format. Only **one team member** submits this report. Please do not forget to include your team members' names and student IDs. One submission per team!
- Your individual report in PDF format. Every team member should submit this report. Please do not forget to include your name and student ID.

8 Demonstration Schedule, Venue and Peer Assessment

- Demonstration is a mandatory component, and the marking process only happens during the demonstration (No demo = No marks!)
- Teams are required to demonstrate the working application via Zoom.
- All team members must be present at the demonstration time slot to answer questions.
- The presentation should start with the architecture of your application as described in your report, along with the design and implementation choices you made.
- Each team has up to 3 minutes to present its architecture.
- This is followed by a demo of the application, as per the examiners' guidelines, for up to 15 minutes, and Q&A. We will announce the demo date and time closer to the due date.
- We will ask you to select a time slot during the allocated hours to join a Zoom meeting.
- As mentioned earlier, this is a group assignment. If one of the team members is experiencing difficulties or has special circumstances, the rest of the team should proceed with the project without that member. This aligns with industry practices where the entire project cannot be delayed due to a team member's departure or absence.
- At the end of the session, teams will conduct peer assessments of their fellow members' contributions and involvement in the project. **Those who do not actively contribute to the group project may be penalised for up to 50% off their group mark.**

This feedback may be gathered through a Google Form. Each team member is required to submit a **confidential report** detailing their role in the project and their experiences collaborating with team members.

These reports will be treated with the utmost confidentiality, along with your final report. The purpose of this evaluation is not to assign blame, but rather to ensure that all team

members have an opportunity to provide comments and to guarantee active participation from everyone.

- We encourage all team members to collaborate effectively to achieve the project's objectives and optimise your team's performance. Should you observe a team member encountering difficulties with their tasks, please offer assistance. If a team member fails to fulfil their assigned tasks or deviates from the project's expectations, we urge you to promptly inform the teaching team. Alternatively, you can utilise this opportunity to bring the matter to our attention.

9. Generative AI Statement

As per the University's [policy](#) on the guidelines and practices pertaining to the usage of Generative AI:

AI & Generative AI tools may be used SELECTIVELY within this assessment.

Where used, AI must be used responsibly, clearly documented and appropriately acknowledged (see Learn HQ).

Any work submitted for a mark must:

- Represent a sincere demonstration of your human efforts, skills and subject knowledge that you will be accountable for (students should be able to understand and modify their code if generated using GenAI tools).
- Adhere to the guidelines for AI use set for the assessment task.
- Reflect the University's commitment to academic integrity and ethical behaviour.

Inappropriate AI use and/or AI use without acknowledgement will be considered a breach of academic integrity.

The teaching team encourages students to apply their own critical thinking and reasoning skills when working on the assessments with an assistant from GenAI. Generative AI tools may produce inaccurate content, which could negatively impact students' comprehension of big data topics.